

Resumen de la Sesión

Bootcamp de JavaScript Avanzado — Semana 1: Callbacks, Event Loop, Closures y Promesas

Fuente: transcripción de audio (grabaciones 2, 3 y 4)

Contexto y objetivo de la sesión

El instructor (antiguo director del bootcamp) presenta la dinámica del curso: cada semana se entrega teoría y ejercicios para practicar de forma autónoma, mientras que la sesión en directo sirve para explicar los conceptos clave antes de que el alumnado se enfrente a la práctica, facilitando así una mejor comprensión y retención.

Mapa del contenido del bootcamp

Semanas	Contenido
1 y 2	JavaScript avanzado: cómo funciona realmente el lenguaje por dentro (callbacks, event loop, closures, promesas, async/await).
3	Maquetación y preprocesadores (nivel profesional, no maquetación básica).
4+	Web Components (base de todos los frameworks) → Frameworks (Angular u otros) → Microfrontales → Performance en aplicaciones web/híbridas.

★ Idea central de la semana

JavaScript se explica "por dentro": no se trata de repasar sintaxis básica, sino de entender el funcionamiento real del motor (single thread, asincronía, memoria/contexto) para poder resolver problemas y superar entrevistas técnicas con seguridad.

1. JavaScript es de un solo hilo ("monohilo")

JavaScript ejecuta una única tarea a la vez (a diferencia de una CPU con varios núcleos que puede procesar en paralelo). Si no se gestiona correctamente esta limitación, el lenguaje puede "saturarse" y bloquear la ejecución.

La **asincronía** es la solución: no crea multihilo real, sino que simula la ejecución en paralelo apoyándose en herramientas que ofrece el navegador (las *Web APIs*), ya que el propio motor de JavaScript sigue siendo monohilo.

★ Punto clave

Monohilo = una tarea cada vez. La asincronía (Web APIs, callbacks, promesas, async/await) es lo que permite que JavaScript parezca hacer varias cosas a la vez sin serlo realmente.

2. Callbacks

Definición correcta: un callback es una función que se pasa como argumento a otra función (no su ejecución). Puede ejecutarse en ese mismo momento o en un momento posterior; su ejecución no depende de sí mismo, sino de la función que lo recibe.

Error común de alumnos: escribir el paréntesis al pasar el callback (`miFuncion()`) ejecuta la función inmediatamente y pasa su resultado, en vez de pasar la función en sí (`miFuncion`).

¿Cuándo se usa un callback?

- Para reaccionar ante un evento que ocurrirá en un momento desconocido (ej. un *listener* de clic).
- Para operaciones costosas que deben ejecutarse en otro momento (ej. una petición *fetch* a una API).

Callbacks síncronos vs. asíncronos

Síncronos	Asíncronos
Se ejecutan en el mismo instante. Ej.: <code>array.map()</code> , que recorre el array y ejecuta el callback al momento.	Se ejecutan cuando ocurre un evento futuro. Ej.: un <i>event listener</i> de clic sobre un botón.

El problema: "Callback Hell"

Cuando se anidan callbacks dentro de callbacks (una función que llama a otra, que llama a otra...), se forma una pirámide de código conocida como *callback hell* (o "pirámide de la perdición"). Es muy difícil de leer y de depurar, y debe evitarse siempre. La solución moderna son las **promesas encadenadas** y **async/await**.

★ Punto clave

Callback = la función en sí, no su ejecución. Nunca anides más de 1-2 niveles de callbacks: si lo haces, es señal de que necesitas promesas o async/await.

3. El Event Loop (bucle de eventos)

Es el mecanismo que permite a JavaScript (monohilo) gestionar tareas asíncronas sin bloquearse. Siempre está "corriendo" en segundo plano, vigilando si hay tareas pendientes por ejecutar.

Piezas del mecanismo

- **Call Stack (pila de ejecución):** donde se apilan y ejecutan las funciones, de arriba abajo, en orden.
- **Web APIs:** funcionalidades que ofrece el navegador (fetch, setTimeout, event listeners, geolocalización, etc.) para delegar tareas fuera del hilo principal.
- **Cola de espera (callback queue):** donde se van colocando los callbacks cuya Web API ya terminó, a la espera de entrar al Call Stack.
- **Event Loop:** revisa constantemente si el Call Stack está vacío; si lo está, mueve el siguiente callback de la cola al Call Stack.

Flujo típico (ejemplo con setTimeout)

1) Se ejecutan las funciones síncronas del script de arriba a abajo. 2) Al llegar a una función asíncrona (p. ej. setTimeout), esta se delega a la Web API correspondiente y el hilo principal continúa sin esperar. 3) Cuando la Web API termina, coloca el callback en la cola de espera. 4) El Event Loop solo mueve ese callback al Call Stack cuando este está completamente vacío.

★ Pregunta típica de entrevista

¿Cuándo puede entrar un callback al Call Stack? Respuesta correcta: **solo cuando el Call Stack está vacío** y le corresponde el turno según el orden de la cola (no en cuanto termina su temporizador, aunque el stack esté ocupado). Este es uno de los conceptos más preguntados en entrevistas técnicas de JavaScript.

4. Closures (clausuras)

Un closure ocurre cuando una función devuelve otra función, y esta función devuelta conserva acceso a las variables de su entorno léxico (el contexto en el que fue creada), aunque la función exterior ya haya terminado de ejecutarse.

Qué NO es un closure: no es una copia congelada de variables globales, ni el valor de `this`, ni simplemente "el resultado final de la función exterior". **Qué SÍ es:** el acceso mantenido a las variables de su entorno léxico, que siguen siendo referenciadas por la función interna.

Ejemplo mencionado: contador

Una función `crearContador()` define una variable interna `valor` y devuelve una función que la incrementa cada vez que se llama. Como esa variable vive dentro del closure, cada llamada recuerda el estado anterior y sigue sumando, sin que se pueda acceder ni reiniciar esa variable desde fuera.

let vs. var (recordatorio)

- **var:** prácticamente en desuso; su alcance es global y puede generar fugas de variables.
- **let / const:** quedan contextualizadas al bloque/función donde se declaran (fundamentales para que los closures funcionen bien).

★ Punto clave

Closure = función interna que "recuerda" y mantiene acceso a las variables de la función que la creó, incluso después de que esa función externa haya finalizado.

5. Promesas y async/await

Una **promesa** representa un resultado que llegará más adelante, y que puede terminar bien o fallar. Tiene tres estados: **pendiente**, **cumplida** (resuelta) o **rechazada**.

Método	Cuándo se ejecuta
.then()	Cuando la promesa se resuelve correctamente.
.catch()	Cuando se produce un error.
.finally()	Siempre, haya éxito o error.
Promise.all()	Espera a que todas las promesas indicadas se resuelvan correctamente.
Promise.race()	Se resuelve con la primera promesa que termine (de varias disponibles).

async / await

async delante de una función indica que esta siempre devolverá una promesa (aunque el return sea un valor directo, se envuelve automáticamente en una promesa).

await pausa la ejecución de esa función asíncrona hasta que la promesa se resuelva. **Importante:** solo bloquea dentro de esa función asíncrona (no bloquea todo el hilo principal), y debe usarse con cuidado porque detiene el flujo hasta obtener el resultado.

★ Punto clave

async/await es "azúcar sintáctico" sobre las promesas: hace el código asíncrono más legible (parece síncrono) evitando el encadenado extenso de .then(). Su gran ventaja frente al callback hell es la claridad y facilidad de manejo de errores con try/catch.

6. Ejemplo práctico comentado en clase

Se analizó una función `cargarUsuario(id, onSuccess, onError)` que realiza un fetch (una Web API) a una API para pedir datos de un usuario:

- **fetch** devuelve una promesa.
- Si la petición se resuelve correctamente, se convierte la respuesta a JSON y se ejecuta el callback `onAccess`.
- Si ocurre un error, se ejecuta el callback `onError`.
- Ambos son callbacks: funciones que se pasan como argumento y que se ejecutarán en un momento

posterior, sin paréntesis al pasarlas (para no ejecutarlas antes de tiempo).

También se repasó el caso de `buscarProducto(id, callback)`, donde el callback podría ser una función como `comprobarStock`, reforzando la idea de definir funciones independientes y pasarlas como parámetro en lugar de anidar lógica.

Resumen de puntos esenciales para recordar

- JavaScript es monohilo: ejecuta una tarea cada vez; la asincronía lo simula mediante Web APIs del navegador.
- Un callback es la función en sí (sin paréntesis), pasada como argumento, que se ejecuta en el momento o después.
- Evitar el callback hell (anidar callbacks): usar promesas encadenadas o `async/await` en su lugar.
- El Event Loop mueve un callback de la cola de espera al Call Stack solo cuando este último está vacío.
- Un closure es una función que conserva acceso a las variables de su entorno léxico, aunque la función que las creó ya haya terminado.
- Usar siempre `let/const` en vez de `var`, para mantener el contexto correctamente delimitado.
- Las promesas tienen 3 estados (pendiente, cumplida, rechazada) y se gestionan con `then/catch/finally` o con `async/await`.
- El Event Loop y los closures son temas recurrentes en entrevistas técnicas de JavaScript.

Nota: este resumen se ha elaborado a partir de la transcripción automática de audio de la sesión en directo del bootcamp; algunos fragmentos originales eran confusos por errores propios de la transcripción y se han interpretado según el contexto técnico.